MagicCode Super-Resolution Software API Reference v1.1

Document Version: v1.1
Header: `mc_interface.h`

1. Overview

The MagicCode Super-Resolution native API is a C-compatible interface for initialization, image processing, parameter control, status query, resource release, and version query. C++ projects should include the header normally; symbols are exported with `extern "C"`.

2. Core Workflow

1. Create and fill `input_param_t`.
2. Call `MC_Init` and keep the returned opaque handle.
3. Call `MC_Process` for processing.
4. Optionally call `MC_Control` with `SET_PARAM` or `QUERY_STATUS`.
5. Call `MC_Uninit` exactly once for each successfully initialized handle.

3. Enumerations

Enum	Values	Description
`cmd_params_e`	`SET_PARAM`, `QUERY_STATUS`	Control command type.
`input_type_e`	`INPUT_BUFFER`, `INPUT_TEXTURE_RGB8Unorm`, `INPUT_TEXTURE_R8Unorm`	Input memory or texture type.
`alg_mode_e`	`HIGH_SPEED_MODE`, `SPEED_MODE`	Public algorithm mode.

`magic_backend_e`
`MAGIC_BACKEND_DEFAULT`, `MAGIC_BACKEND_X86`, `MAGIC_BACKEND_NEON`, `MAGIC_BACKEND_METAL`, `MAGIC_BACKEND_OPENGL`, `MAGIC_BACKEND_OPENGLES`, `MAGIC_BACKEND_VULKAN`
Runtime backend selector.
`log_level_e`
`NONE`, `ERROR`, `WARNING`, `INFO`, `DEBUG`
Runtime log verbosity.

4. Structures

4.1 input_param_t

Field	Description
`input_type`	Input type. CPU backends use buffer input; GPU backends use texture input.
`model_path[256]`	Path to model file. Maximum effective path length is 255 characters plus null terminator.
`width`, `height`	Input width and height. Valid range: `64` to `4032`.
`scaler_factor`	Requested scale. Valid range: `1.0` to `8.0`; actual support depends on backend/model.
`alg_mode`	`HIGH_SPEED_MODE` or `SPEED_MODE`.
`num_threads`	CPU thread count, valid range 1 to 8. Multi-threading takes effect when height is at least 256.
`log_level`	Log verbosity.
`backend`	Runtime backend selector.

4.2 control_param_t

Used with `MC_Control(..., SET_PARAM, ...)` to update width, height, scale, mode, and model path. Reinitialization may occur

internally when parameters require it.

4.3 output_status_params_t

Used with `MC_Control(..., QUERY_STATUS, ...)`. It returns input dimensions, output dimensions, scale, algorithm mode, input type, backend, thread count, GPU time, and latest error code.

5. Functions

Function	Purpose	Return
`void* MC_Init(input_param_t* param)`	Initializes the SR engine and creates a handle.	Opaque handle on success; NULL on failure.
`int MC_Process(void* handle, void* image_in, void* image_out)`	Runs super-resolution on one input buffer or texture.	`0` on success; non-zero on failure.
`int MC_Control(void* handle, cmd_params_e cmd, control_param_t* ctrl, output_status_params_t* output)`	Sets parameters or queries status.	`0` on success; non-zero on failure.
`int MC_Uninit(void* handle)`	Releases resources for a handle.	`0` on success; non-zero on failure.
`char* MC_GetVersion(void)`	Returns static version string.	Non-null version string pointer. Do not free.

6. Minimal Native C Example

The following example shows the basic lifecycle for CPU buffer input. It is intended as a compact reference for API usage. Production code should add project-specific memory management, image format conversion, logging, and error handling.

```
#include "mc_interface.h"
#include <stdio.h>
#include <string.h>
```

```
int run_magic_sr_y8(const char *model_path,
                    unsigned char *input_y,
                    unsigned char *output_y,
                    unsigned int width,
                    unsigned int height)
{
    input_param_t param;
    memset(&param, 0, sizeof(param));

    param.input_type = INPUT_BUFFER;
    strncpy(param.model_path, model_path,
sizeof(param.model_path) - 1);
    param.width = width;
    param.height = height;
    param.scaler_factor = 2.0f;
    param.alg_mode = HIGH_SPEED_MODE;
    param.num_threads = 4;
    param.log_level = MAGIC_LOG_INFO;
    param.backend = MAGIC_BACKEND_NEON; /* Use
MAGIC_BACKEND_X86 on x86_64 CPU builds. */

    printf("MagicSR version: %s\n", MC_GetVersion());

    void *handle = MC_Init(&param);
    if (handle == NULL) {
        printf("MC_Init failed\n");
        return -1;
    }

    int process_ret = MC_Process(handle, input_y,
output_y);
    if (process_ret != 0) {
        printf("MC_Process failed: %d\n",
process_ret);
        MC_Uninit(handle);
        return process_ret;
    }
```

```
    output_status_params_t status;
    memset(&status, 0, sizeof(status));
    int query_ret = MC_Control(handle, QUERY_STATUS,
NULL, &status);
    if (query_ret == 0) {
        printf("output=%ux%u backend=%d
error=0x%08x\n",
               status.output_width,
               status.output_height,
               status.backend,
               status.error_code);
    }

    int uninit_ret = MC_Uninit(handle);
    return uninit_ret;
}
```

6.1 Example Notes

- Use `MAGIC_BACKEND_NEON` on ARM64 CPU builds and `MAGIC_BACKEND_X86` on supported x86_64 CPU builds.
- For GPU backends such as Metal, OpenGLES, or Vulkan, set a texture input type and pass valid backend-specific texture handles to `MC_Process`.
- The model file must match the selected backend, algorithm mode, and scale configuration.
- Always call `MC_Uninit` for a successfully created handle, including error paths after initialization.

7. Error Handling

Most API failures are represented by negative error codes. Refer to the Error Code Manual for the complete list. Common categories include initialization errors, processing errors, control errors, uninitialization errors, memory errors, and reporting errors.

8. Threading and Resource Notes

- Do not use a handle after `MC_Uninit`.
- Do not call `MC_Uninit` multiple times on the same

handle.

- Use separate handles for independent streams unless your integration has explicitly serialized access.
- Copy or consume output data before subsequent calls if your integration uses reusable internal buffers.